

5-Bar SCARA v2 Onboarding Package

M2

Encoder + Sensor Fusion Edition | Engineering Roadmap

FIRMWARE LEAD — KINEMATIC NODE

1 | ROLE & OBJECTIVE

As **Firmware Lead — Kinematic Node (M2)**, you are the computational brain of the robot arm. Your mission spans two critical phases: **Phase 2** — build the closed-form IK solver, 2D collision monitor, and interrupt-driven state machine (open-loop baseline); and **Phase 2B** — layer quadrature encoder decoding, a closed-loop position correction engine, and stall detection onto that foundation. You also serve as reviewer for M1's mechanical output.

Primary Mission: Deliver firmware on the **ESP32 (dual-core)** that reliably accepts FusionPackets from M4, resolves inverse kinematics, drives AccelStepper motors, and uses encoder feedback to verify and correct each commanded position — all in real time without blocking the main loop.

2 | HARDWARE & COMPONENTS REQUIRED

Component	Specification / Notes	Qty
ESP32 Dev Board (dual-core)	Main kinematic processing node; must have 4 interrupt-capable GPIO free	1
DRV8825 Stepper Drivers	One per motor; set microstepping to 1/16 or 1/32	2
NEMA 17 Stepper Motors	As mounted by M1 on SCARA frame	2
Rotary Encoders (mounted by M1)	LPD3806 600PPR or AS5600; A/B signals on GPIO 34,35,36,39	2
Power Supply (12V, >3A)	For DRV8825 VMot; separate from ESP32 3.3V rail	1
Logic Level Converter	If encoder signals are 5V — convert to 3.3V for ESP32	As needed
Oscilloscope / Serial Monitor	Verify ISR counts, step pulses, correction loop behavior	1
USB-to-Serial Adapter	For flashing and Serial monitor on ESP32	1

3 | SOFTWARE & PLATFORMS REQUIRED

Tool / Library	Purpose	Source
Arduino IDE 2.x or PlatformIO	Primary firmware development environment	arduino.cc / platformio.org

Tool / Library	Purpose	Source
Espressif ESP32 Board Package	FreeRTOS, IRAM_ATTR, hardware timers, noInterrupts()	Board Manager
AccelStepper Library	Non-blocking stepper motor control (.run() in main loop)	Library Manager
ESP-NOW (built-in)	Receive FusionPacket from M4's vision bridge	Included in ESP32 core
FreeRTOS (built-in)	Run stall detection check_stall() as a timed task	Included in ESP32 core
Git	Version control; review M1's config.h before Phase 2	git-scm.com
Python + matplotlib (optional)	Plot encoder count vs. step count during calibration	pip install matplotlib

4 | AI INTEGRATION STRATEGY

Your work is mathematically dense (IK geometry, ISR timing, FreeRTOS task scheduling). Use AI as a pair-programmer for derivation verification, ISR debugging, and test generation:

Prompt 1 — Verify your IK math (law of cosines):

"I have a 5-bar parallel SCARA with Motor 1 at $P1=(-d/2, 0)$ and Motor 2 at $P2=(+d/2, 0)$. Link lengths are $L1=118.4$, $L2=120.1$, $L3=117.9$, $L4=119.6$, $d=95.0$ (all mm). Derive the closed-form elbow-out IK solution for both chains using law of cosines. Show the atan2+alpha formulation for theta1 and theta2. Clamp cos values to [-1,1]. Return as Arduino C++ function IKResult solve_ik(float x, float y)."

Prompt 2 — Debug quadrature ISR jitter:

"My ESP32 quadrature decoder ISR uses the QEM[16] lookup table. At high motor speeds I see occasional count errors of +2 or -2 per revolution. The encoder is 600PPR. Diagnose possible causes: bounce, cache miss latency, simultaneous A+B edge timing. How does IRAM_ATTR help? What is the maximum safe RPM before ISR misses edges?"

Prompt 3 — Generate TEST 2-A validation code:

"Generate an Arduino test sketch that validates my IK solver with 20 random (x,y) workspace points. For each point: call solve_ik(), then forward-kinematics to recover (x,y), then compute Euclidean error. Print PASS if all errors < 0.5mm, FAIL otherwise. Include the forward kinematics for a 5-bar parallel robot given theta1, theta2, L1-L4, d."

Prompt 4 — State machine review:

"Review this RobotState enum and main loop pattern for a pick-and-place robot: [paste your state machine code]. Identify any blocking call that would starve AccelStepper.run(), any state transition that could skip a state under fast packet arrival, and any missing FAULT recovery path."

5 | BASIC ARCHITECTURE & CONNECTIONS

Input From	Your Processing	Output To
M1: config.h (L1-L4, d, GPIO, PPR)	IK solver reads all geometric constants at compile time	M4: acknowledgment + motor state

Input From	Your Processing	Output To
M4: FusionPacket (X,Y,Vx,Vy,confidence)	Parse packet → IK → collision check → moveTo()	DRV8825 → NEMA 17 motors
Encoders (A/B pulses via ISR)	Quadrature decode → angle comparison → correction move	M5: stall alerts via Serial
M1: home position marker	Zero encoder counts at startup homing routine	M5 reviewer: Phase 2B gate
M3: velocity hints (Vx,Vy)	Pre-position if confidence > 180 threshold	Reduced pick latency

Core Loop: ESP-NOW ISR (IRAM_ATTR) receives FusionPacket → main loop reads g_target → solve_ik() → check_collision() → moveTo() → motor1.run()/motor2.run() → apply_correction() → check_stall() [50ms hardware timer]

6 | TIMELINE & MILESTONES

[DAY 1]

- Pull config.h from M1 (or Git baseline) — read all geometric constants
- Set up PlatformIO project with ESP32 target and AccelStepper dependency
- Stub out IKResult struct and solve_ik() function signature
- Coordinate with M4 on FusionPacket struct definition (v2 version)

[DAYS 2–5 (Phase 2 — IK Solver & State Machine)]

- Implement IK solver: law of cosines, atan2 + alpha, elbow-out branch for both chains
- Implement 2D collision monitor using cross-product segment intersection
- Build state machine: IDLE → HOMING → MOVING_TO_PICK → PICKING → MOVING_TO_DROP → DROPPING → FAULT
- TEST 2-A: IK round-trip error < ±0.5mm on 20 random workspace points
- TEST 2-B: Collision detector 100% accuracy on 20 known test cases
- TEST 2-C: State machine driven by Serial mock packets — full IDLE→PICK→DROP→IDLE cycle
- TEST 2-D: Out-of-workspace target silently rejected
- M1 reviews Phase 2 — obtain sign-off

[DAYS 6–8 (Phase 2B — Encoder Closed-Loop)]

- Implement isr_enc1() and isr_enc2() with IRAM_ATTR and QEM[16] lookup table
- Implement attach_encoders() with CHANGE interrupts on all 4 encoder pins
- Thread-safe get_enc1_angle_deg() and get_enc2_angle_deg() with noInterrupts()
- Implement apply_correction(): proportional correction loop, max 3 attempts
- Implement check_stall(): 50ms FreeRTOS timer, STALL_COUNT_THRESHOLD=20 counts
- TEST 2B-A: Encoder direction — +90° command → count = +ENC_CPR/4
- TEST 2B-B: Correction triggered when linkage manually held
- TEST 2B-C: FAULT triggered within 700ms of shaft block
- TEST 2B-D: 10 pick positions, encoder error < ±1.5 deg each
- TEST 2B-E: Zero drift in 30 seconds idle
- M5 reviews Phase 2B — obtain sign-off

7 | ESCALATION MATRIX

Issue / Blocker	First Contact	Second Contact
IK returns invalid for reachable points	Self-debug with AI Prompt 1; check D/2 offset	M1 — re-verify physical L values
Encoder counts wrong direction	Swap A/B wires; verify QEM table orientation	M1 — check encoder physical mounting
ISR misses counts at high speed	Verify IRAM_ATTR; reduce max motor speed	M5 — test at reduced speed first
AccelStepper blocks (loop starves)	Verify motor1.run() in main loop, never delay()	M4 — check if comms ISR is blocking
Stall detection false triggers	Increase STALL_COUNT_THRESHOLD; check loose coupling	M5 — review threshold tuning
FusionPacket struct size mismatch	M4 — sync struct definition with static_assert	M6 — baseline config arbitration
Phase 2B gate blocked by M5	M5 directly; share Serial logs for async review	M6 for escalation

You are writing the nervous system of this robot. Every ISR, every state transition, every correction loop you build with precision makes the whole team's work reliable. Trust your math, test every path, and never merge code that hasn't passed its gate. ■